

Aviator Design Document

David Thoe, Joshua Kim, Zeke Ulrich, Juan Vargas

November 22, 2025

GTA: Ruihang Ni
Professor: Minghao Qi



Figure 0.0.1: [Caption]

Contents

1	Introduction	9
1.1	Executive Description	9
1.2	User Stories	9
2	Design Requirements	10
2.1	Requirements	10
2.2	Factors Influencing Requirements	11
2.2.1	Public Health, Safety, and Welfare	11
2.2.2	Cultural Factors	11
2.2.3	Social Factors	11
2.2.4	Environmental Factors	11
2.2.5	Economic Factors	11
3	System Overview	12
3.1	System Block Diagram	12
3.2	System Activity Diagram	13
3.3	System Mechanical Design (Extra Credit)	14
3.4	Integration Approach	16
3.5	System Photographs	17
4	Subsystems	19
4.1	Subsystem 1: Processing	20
4.1.1	Subsystem Diagrams	20
4.1.2	Specifications	20
4.1.3	Subsystem Interactions	20
4.1.4	Core ECE Design Tasks	21
4.1.5	Parts	21
4.1.6	Algorithm	21
4.1.7	Theory of Operation	22
4.1.8	Specifications Measurement	22
4.1.9	Standards	23
4.2	Subsystem 2: Power Management	24
4.2.1	Subsystem Diagrams	24
4.2.2	Specifications	24
4.2.3	Subsystem Interactions	25
4.2.4	Core ECE Design Tasks	25
4.2.5	Schematics	25
4.2.6	Parts	25
4.2.7	Finalized Parts	26

4.2.8	Algorithm	26
4.2.9	Theory of Operation	26
4.2.10	Specifications Measurement	27
4.2.11	Standards	27
4.3	Subsystem 3: Text Display	29
4.3.1	Subsystem Diagrams	29
4.3.2	Specifications	29
4.3.3	Subsystem Interactions	29
4.3.4	Core ECE Design Tasks	30
4.3.5	Schematics	30
4.3.6	Parts	31
4.3.7	Algorithm	32
4.3.8	Theory of Operation	33
4.3.9	Specifications Measurement	33
4.3.10	Standards	34
4.4	Subsystem 4: Network and Communications	35
4.4.1	Subsystem Diagrams	35
4.4.2	Specifications	35
4.4.3	Subsystem Interactions	36
4.4.4	Core ECE Design Tasks	36
4.4.5	Schematics	36
4.4.6	Parts	37
4.4.7	Algorithm	37
4.4.8	Theory of Operation	39
4.4.9	Specifications Measurement	39
4.4.10	Standards	39
5	PCB Design	41
5.1	PCB Schematics	41
5.2	PCB Layout	42
6	Final Status of Requirements	43
7	Team Structure	45
7.1	Team Member 1	45
7.2	Team Member 2	45
7.3	Team Member 3	46
7.4	Team Member 4	46
8	Bibliography	48
9	Appendices	49

List of Figures

0.0.1 [Caption]	2
3.1.1 Aviator Block Diagram	12
3.2.1 Aviator Activity Diagram	13
3.3.1 Aviator Mechanical Design	15
3.5.1 Photograph of assembled system	18
4.1.1 Processing subsystem Block Diagram	20
4.2.1 Power subsystem Block Diagram	24
4.2.2 [Schematic Name]	28
4.3.1 Display subsystem Block Diagram	29
4.3.2 Display Driver Schematic (1/2)	30
4.3.3 Display Driver Schematic (2/2) and features	31
4.4.1 Network subsystem Block Diagram	35
4.4.2 [Schematic Name]	40
5.1.1 PCB Schematic	41
5.2.1 PCB Layout	42

List of Tables

1 Revision Log 7

Revision Log

Date	Revision	Changes	Initials
05/03/2024	v0.1	Initial Release	(PU)
09/18/2025	v0.2	First Submission	(ZU)
10/05/2025	v0.3	Second Submission	(DT)
11/11/2025	v0.4	Corrections, Updates	(DT)

Table 1: Revision Log

Glossary

- **API** Application Programming Interface.
- **ADS-B** Automatic Dependent Surveillance–Broadcast
-

1 Introduction

1.1 Executive Description

Retro-themed aircraft information display.

1.2 User Stories

The Long-Time Aviation Hobbyist

As a long-time aviation hobbyist who has spent years tracking flights through phone apps, I'm tired of paying for subscriptions just to unlock basic features. I want a device that gives me real-time flight information without hidden costs, while also providing a tactile, nostalgic experience that reminds me of classic aviation boards. By having the Aviator on my desk, I can finally stay connected to the aviation world without feeling like I'm paying a premium for something that should be standard and accessible.

The Casual Aviation Enthusiast

As someone with a general interest in aviation, I don't need a full cockpit-level tracker, but I do want something that feels engaging and easy to use. Standard apps are flat, cluttered, and frankly too much for a novice like myself, but the Aviator project makes flight tracking simple, physical, and fun. I can glance at the board, see arrivals and departures, and feel connected to the aviation scene effortlessly. The setup process was simply plug and play. Additionally, I don't have to pay a dime for the product. For me, it's about accessibility and enjoying aviation in a personal, low-effort, high-impact way.

The Purdue ECE Student

As a Purdue ECE student, I'm drawn to the Aviator not only as a hobby project that I can tinker with, but also as a nod to Purdue's deep aviation legacy. It's inspiring to own a piece of tech that bridges my academic interests in circuits and embedded systems with Purdue's reputation in aerospace. I want a tracker that feels hands-on, customizable, and personal—something that makes me feel part of both my field of study and Purdue's aviation history every time I glance at it. Given the nature of the project and its ability to be completed by an individual excites me, as it gives me the stepping stone I needed to start tracking flights.

2 Design Requirements

2.1 Requirements

1. The device must display accurate information.
2. The display must not interfere with user's well-being by, for example, displaying at excessive luminosity, updating rapidly in a distracting manner, or being excessively bulky.
3. The device must not infringe on any person's reasonable expectation of privacy.
4. The device must be language-agnostic wherever possible.
5. The device must be responsive and intuitive.
6. The device must have robust error handling and recovery.
7. The physical device should be easily replicated with widely available parts.
8. The code for the device must be open-source and well-documented.
9. The device should be as durable and environmentally friendly as possible so as not to contribute to e-waste.
10. The device must not contribute to noise or visual pollution of any space.
11. The device must be energy-efficient.
12. The device must minimize construction and recurring costs.
13. The device must not infringe on right to repair.
14. The device must mount and dismount without damage to vertical surfaces.

2.2 Factors Influencing Requirements

2.2.1 Public Health, Safety, and Welfare

1. User well-being
2. Privacy

2.2.2 Cultural Factors

1. Language differences
2. Ease of use

2.2.3 Social Factors

1. Ease of replication
2. Open-source and documentation

2.2.4 Environmental Factors

1. Environmental friendliness and e-waste
2. Noise and visual pollution
3. Energy efficiency

2.2.5 Economic Factors

1. Cost
2. Repairability

3 System Overview

3.1 System Block Diagram

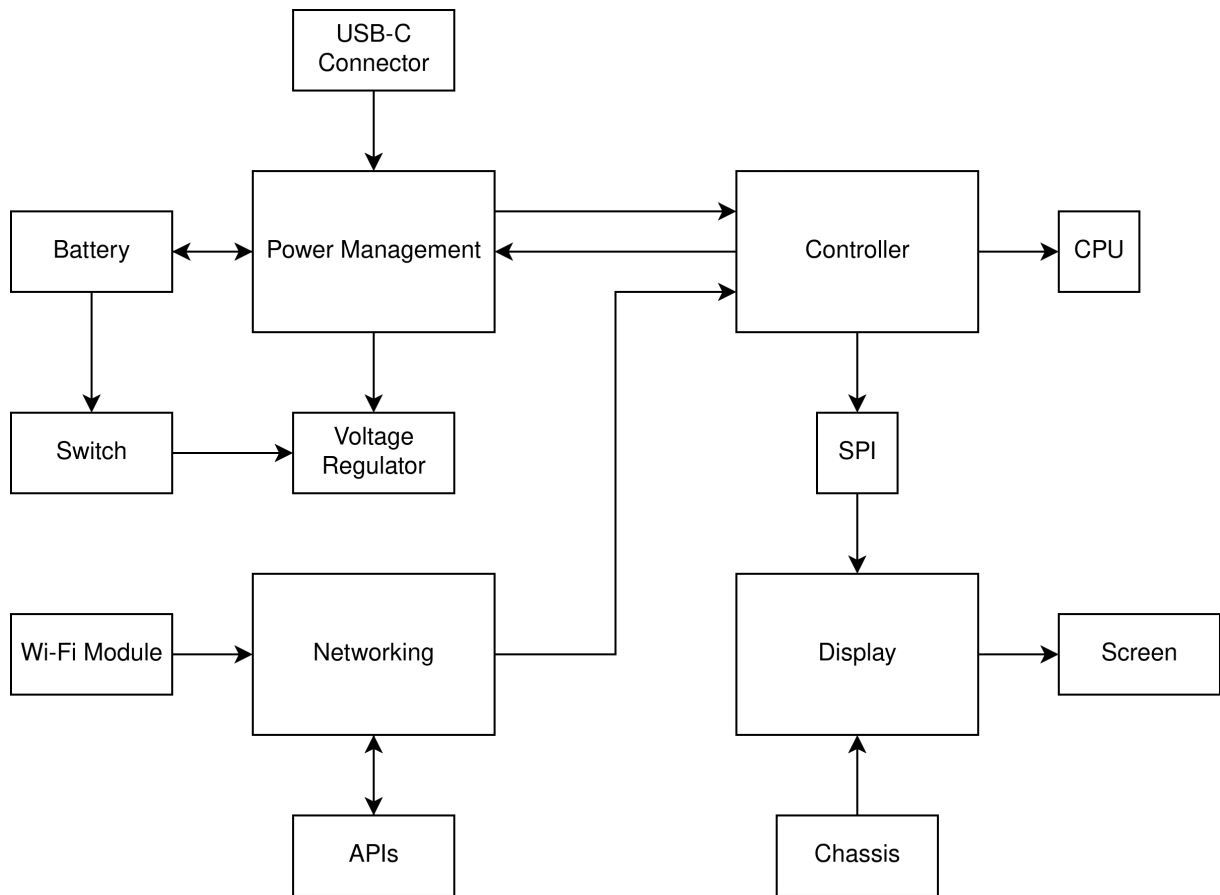


Figure 3.1.1: Aviator Block Diagram

3.2 System Activity Diagram

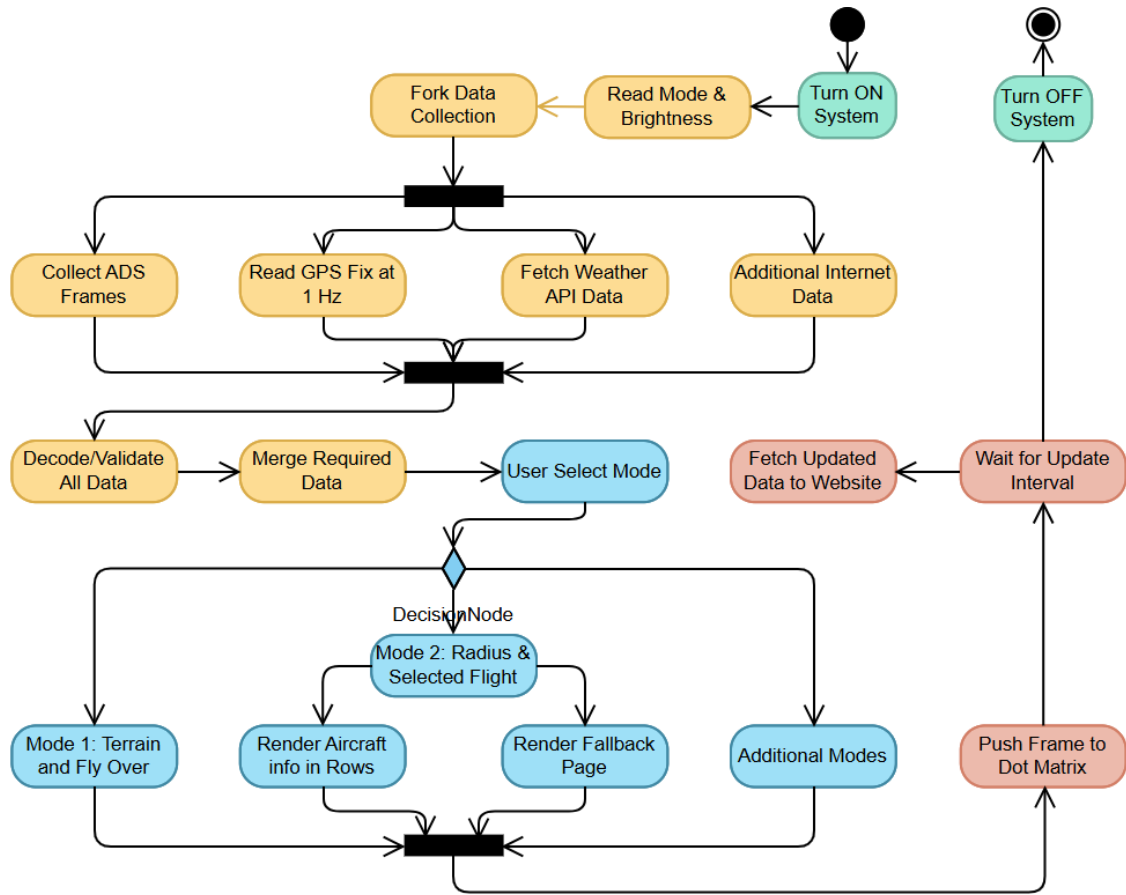


Figure 3.2.1: Aviator Activity Diagram

3.3 System Mechanical Design (Extra Credit)

[DD3+]



Figure 3.3.1: Aviator Mechanical Design

3.4 Integration Approach

The system integrates an ADS-B software-defined radio (SDR), a cloud data pool, a local processing subsystem, a display module, and a power management subsystem. The ADS-B SDR publishes raw flight data to the cloud pool; the processing subsystem ingests that cloud stream plus external APIs and user inputs, normalizes and filters the data, and forwards compact display updates to the display module. The power management subsystem supplies energy to the processor and display and includes a battery backup to preserve state and reduce startup latency.

Design priorities are resilience and graceful degradation: if network connectivity is lost the device continues to show local time and the last cached flight/weather state; if power to the device is interrupted the ADS-B transmitter can still upload to the cloud independently. Data is cached locally and keyed by timestamp so that, on reconnection, the processor can quickly reconcile and display missed updates. Interfaces are defined around simple, versioned JSON payloads and REST/WebSocket endpoints to minimize coupling and simplify testing. Latency, cache expiration, and failure modes are documented per interface to guide integration testing.

3.5 System Photographs

[**DD3+**] [Photograph of assembled system, intended to highlight user interaction / controls. If system is split into multiple parts, show a composite of more than one photograph with all key user interactions / controls.]



Figure 3.5.1: Photograph of assembled system

4 Subsystems

4.1 Subsystem 1: Processing

4.1.1 Subsystem Diagrams

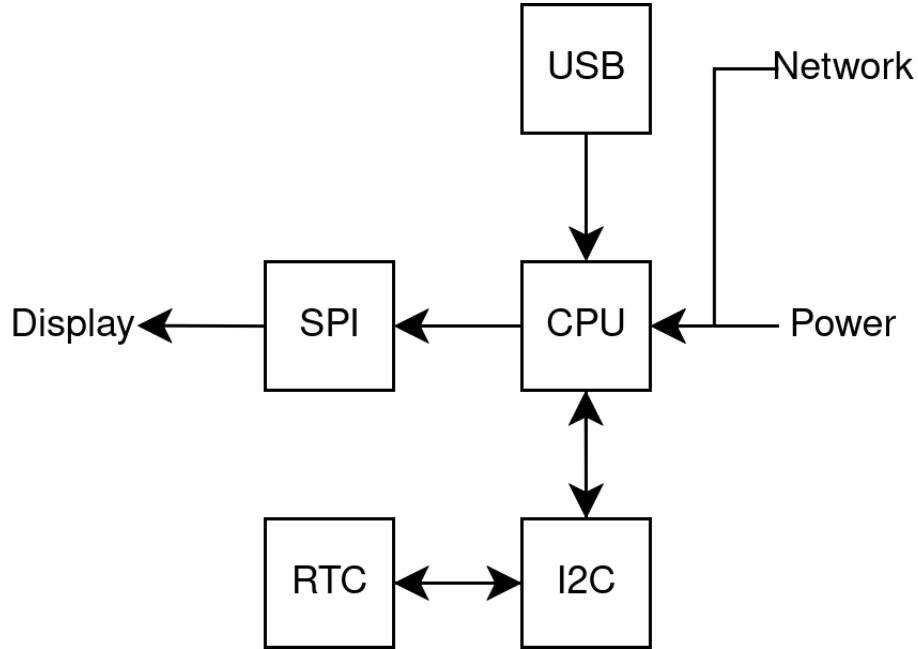


Figure 4.1.1: Processing subsystem Block Diagram

4.1.2 Specifications

1. Clock drift $\leq \pm 1 \frac{\text{sec}}{\text{day}}$
2. After network interrupt, system must reconnect within 120 seconds upon network resumption.
3. System must run with zero memory leaks.
4. System must never display text overrunning borders.

4.1.3 Subsystem Interactions

The core computer interfaces with all other subsystems. The battery/power management unit supplies it with power. Running processes direct and receive information from the network module. It communicates with the digital dot matrix display via SPI according to display drivers on the controller.

4.1.4 Core ECE Design Tasks

- **ENGR 16100:** Teamwork & project documentation.
- **CS 15900:** Fundamentals of programming.
- **ECE 36200:** PCB design and embedded software development.

4.1.5 Parts

- ESP32-S3
- DS3231
- PCB

4.1.6 Algorithm

```
initialize clock , network , display , location
DATA = {}

always
    wifi keep alive
    error handling
    if power button pressed
        save and shut down

every minute
    update display time

    read ADS-B sensor
    if ADS-B valid
        add to DATA
        if internet connected
            upload ADS-B data
        else
            cache ADS-B data
    else
        if internet connected
            make API call for flight data
            add to DATA
        else
            add warning ADS-B out of date to DATA
```

```

    if battery powered
        update battery level in DATA

    update time, weather in DATA
    convert DATA to pixel buffer
    push pixel buffer to display
every hour
    sync with network time
    make API call for weather

```

4.1.7 Theory of Operation

The processing subsystem is responsible for coordinating all other subsystems and serving as the “brain” of the Aviator device. At startup, the ESP32-S3 initializes its real-time clock, connects to the Wi-Fi network, and configures the SPI interface for display communication. During normal operation, the processor maintains a persistent network connection to ensure timely access to APIs.

Every minute, the processor updates the displayed time, retrieves the latest flight information from the network subsystem, and parses the response into a standardized data structure. If the device is battery-powered, it also queries the power management subsystem for voltage information and appends this to the displayed data. The processor then converts the compiled information (time, weather, and flight updates) into a pixel buffer and transmits it to the display subsystem.

On an hourly basis, the processor synchronizes the real-time clock with network time servers and fetches weather data for local context. Error handling routines ensure that loss of network, corrupted data, or power fluctuations do not crash the system; instead, the subsystem retries API calls or falls back to cached data, with a warning that the data is out of date.

4.1.8 Specifications Measurement

1. *Clock drift* $\leq \pm 1 \frac{\text{sec}}{\text{day}}$.

We measured drift over a period of two days with the following script:

```

import time, board, rtc, wifi, socketpool, adafruit_requests
pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, None)
r = requests.get("http://worldtimeapi.org/api/ip")
ref = r.json()['unixtime']

```

```

r.close()
dev = rtc.RTC().datetime
dev_epoch = int(time.mktime(dev[0:6] + (0,0,0)))
print("offset_s", dev_epoch - ref)

```

Reported drift was 1.44 seconds, or 0.72 seconds per day, within tolerance.

2. *After network interrupt, system must reconnect within 120 seconds upon network resumption.*

This was tested by disabling the Wi-Fi hotspot to which the device was connected, waiting 60 seconds, and reabling. Device reconnected in 50 seconds. By design, the device will always try to reconnect once a minute if disconnected.

3. *System must run with zero memory leaks.*

We tested this by enabling heap logging via the `gc` module and tracking over three days while device was in normal operation. Free memory was observed to not have a monotonically decreasing trend beyond noise; upon initialization, free memory was reported as 8274912 and after three days free memory was still reported as 8274912.

4. *System must never display text overrunning borders.* This is accomplished with bounds checking and scrolling text that overflows bounding box. Tested with random strings up to 65 characters long.

4.1.9 Standards

- **IPC-2221:** Governs PCB trace width, spacing, creepage/clearance, via rules, grounding, etc.
- **IPC-A-610:** Covers soldering quality and workmanship.
- **RFC 5905:** Protocol for syncing ESP time to internet.

4.2 Subsystem 2: Power Management

4.2.1 Subsystem Diagrams

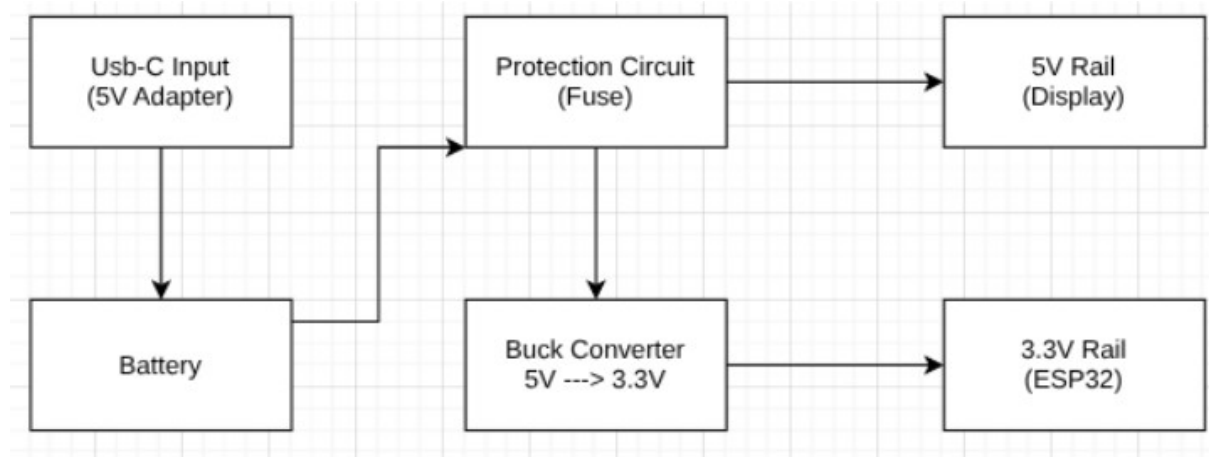


Figure 4.2.1: Power subsystem Block Diagram

4.2.2 Specifications

1. Input Voltage: 5 V from USB-C adapter.
2. Backup Source: 3.7 V Li-ion battery (at least 8000 mAh)
3. Output Rails:
4. 5 V for Display (load up to 3 A)
5. 3.3 V for ESP32 (load up to 1 A)
6. Voltage Accuracy: $\pm 5\%$
7. Average Power Consumption: $\leq 8\text{ W}$
8. Efficiency: 85% for buck conversion; 80% charge/discharge.
9. Protection: fuse (3 A hold)
10. Thermal Limit: surface temperature under $60\text{ }^{\circ}\text{C}$ under full load.

4.2.3 Subsystem Interactions

1. With Processing Subsystem (Zeke): Provides regulated 3.3 V rail to ESP32
2. With Display Subsystem (David): Delivers 5 V rail and handles power demand spikes during high-brightness operation.
3. With PCB Integration (Zeke and Joshua): Power traces routed centrally from power management module on PCB, decoupling capacitors placed near load headers to reduce noise
4. With Networking Subsystem (Juan): Shares 3.3 V supply for network module and provides stable voltage during transmit peaks.

4.2.4 Core ECE Design Tasks

- **ECE 25500:** Electronic Devices and Design Lab: Used to design buck-boost converter and protection circuits
- **ECE 27000:** Digital Systems: Interfaces battery and power-good signals with ESP32 logic pins
- **ECE 36900:** Power Electronics: Guides design of buck and buck-boost stages and load sharing
- **ECE 49595:** Embedded Hardware Design: Applied to joint PCB work with Zeke

4.2.5 Schematics

[Type here **DD2+**]

4.2.6 Parts

- USB-C 5 V input module
- Li-ion battery pack (3.7 V, 8000 mAh)
- Buck-boost regulator IC (for 3.3 V and 5 V rails)
- Battery charging IC (USB-C PD or simple Li-ion charger).
- Basic protection devices (fuse, MOSFET switch, TVS diode).
- (Optional) Solar panel (5–20 W) + MPPT controller IC.

4.2.7 Finalized Parts

- USB-C input controller: STUSB4500 (PD sink, 5 V mode)
- Buck-boost regulator: TI TPS63070 (3.3 V / 5 V output from battery or adapter)
- Battery charging IC: TP4056 (simple Li-ion charger for 5 V input)
- Li-ion battery pack: 3.7 V 8000 mAh flat pack with PCM protection
- Fuse: resettable polyfuse (3 A hold)
- TVS diode: SMBJ5.0A (for transient suppression)
- MOSFET switch: P-channel AO3407A (for reverse protection and load control)
- Connectors / Headers: keyed 5 V and 3.3 V output headers with test pads

4.2.8 Algorithm

- Initialization: detect input source (USB-C present or battery only).
- If USB-C is present: supply system load directly and charge battery at regulated current.
- If USB-C is removed: automatically switch to battery through PowerPath MOSFET (ideal-diode behavior).
- Monitor: ESP32 reads battery voltage and power-good signal periodically.
- Low-voltage event: ESP32 alerts display and reduces brightness to conserve power.

4.2.9 Theory of Operation

The power management subsystem accepts a 5 V input from a USB-C adapter as its primary source. When connected, the charger IC diverts current both to the system load and to the battery for charging. A buck-boost converter regulates voltage for both 5 V and 3.3 V rails regardless of whether the system is running on the adapter or battery. If the adapter is unplugged, the PowerPath MOSFET automatically connects the battery to the load without noticeable interruption (≤ 100 ms). Protection devices (fuse and TVS diode) guard against short circuits and voltage spikes. The ESP32 monitors the battery status and can display charging or battery mode indicators.

4.2.10 Specifications Measurement

[**DD3+** Every specification here should match the specification above.]

1. [Copy specification here.]
[Explain the specification here. Add photoes if necessary.]

4.2.11 Standards

- **USB-C Power Delivery Specification:** ensures safe and standard input power.
- **IEEE 1625:** covers battery system reliability for portable electronics.
- **UL 2054:** safety standard for rechargeable batteries in consumer devices.
- **IEC 61215:** (Optional solar) performance and reliability for PV modules.



Figure 4.2.2: [Schematic Name]

4.3 Subsystem 3: Text Display

4.3.1 Subsystem Diagrams

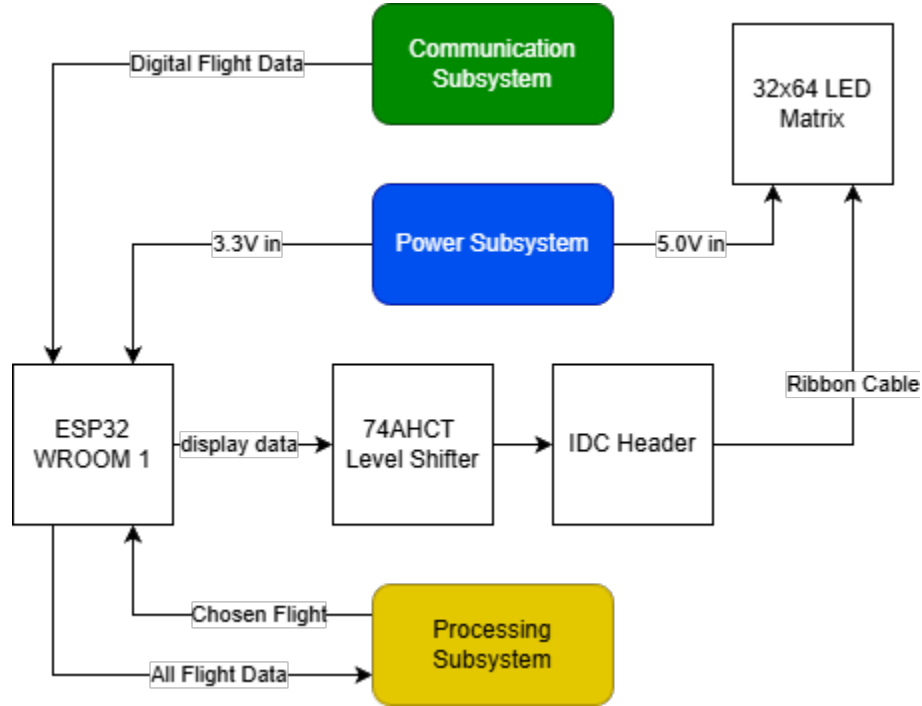


Figure 4.3.1: Display subsystem Block Diagram

4.3.2 Specifications

1. Feature a 2048-pixel Dot Display (64 x 32px)
2. Physical dimensions of housing within 5% of 64 x 128 x 256 mm.
3. No noticable screen tear (refresh rate $\geq$ 60Hz)

4.3.3 Subsystem Interactions

The display subsystem links the main PCB and the LED Matrix. This subsystem will interface all subsystems on and the main PCB. It will receive data from the network and communications subsystem, from the main PCB and processing subsystem it will receive commands regarding the format of the display, interpret the commands, and display the relevant air traffic data.

4.3.4 Core ECE Design Tasks

- **ECE 27000:** Provides a solid foundation in logic circuits. Helps for designing registers, data paths, logic that drives the LED panel.
- **ECE 25500:** Provides a base understanding of transistors, amplifiers, and fundamentals into I-V behavior.
- **ECE 40862:** Teaches valuable STM concepts such as programming, interrupts, and DMA which are needed to drive the display controller efficiently.

4.3.5 Schematics

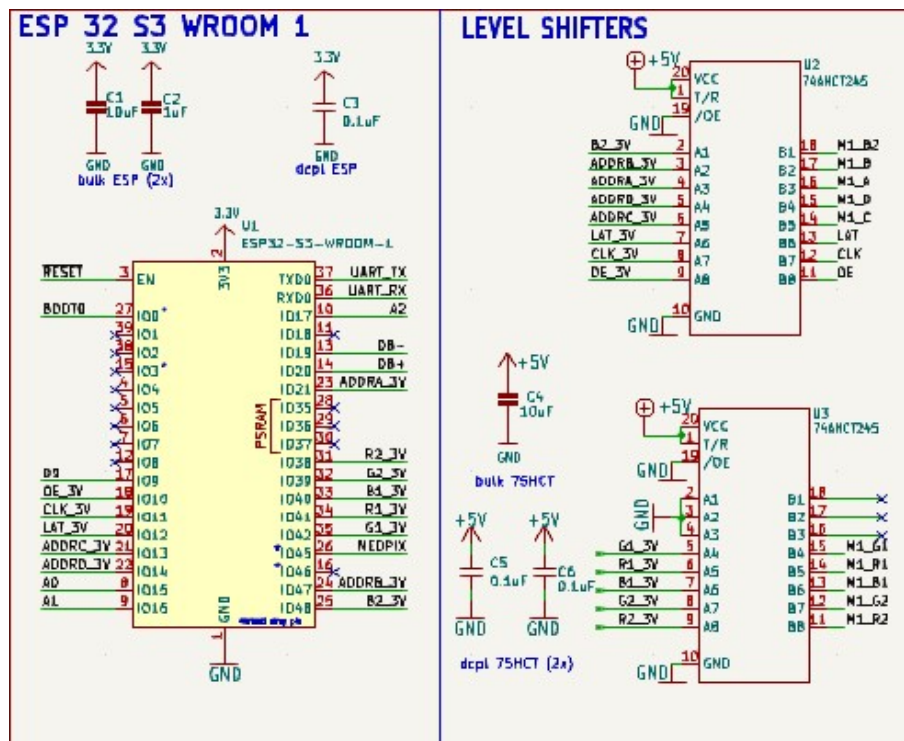


Figure 4.3.2: Display Driver Schematic (1/2)

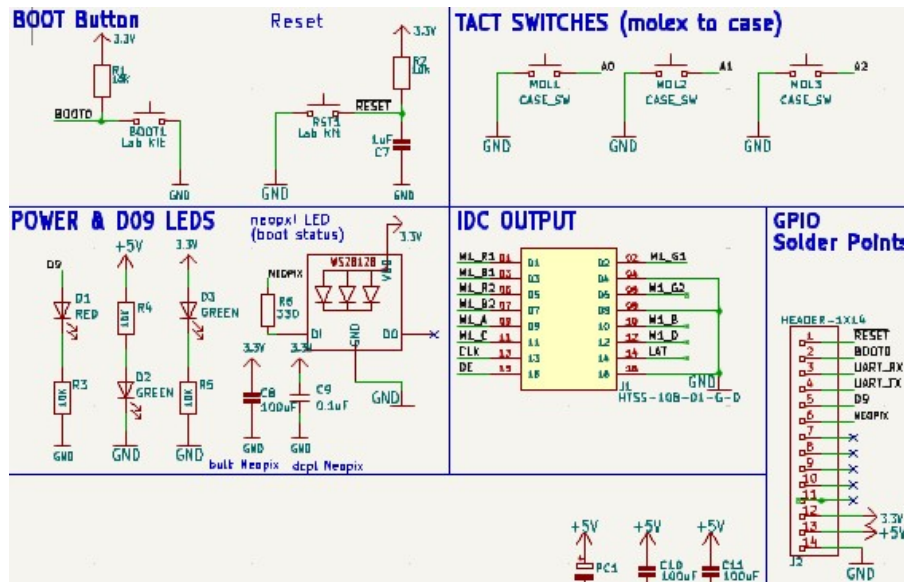


Figure 4.3.3: Display Driver Schematic (2/2) and features

4.3.6 Parts

- LED Matrix (ADAFruit, 32x64, P4)
 - 02x08 Pin header (IDC Connection)
- Resistors:
 - 330
 - 10k (5)
- Capacitors
 - 0.1uF (0603) (3)
 - 1uF (0805)
 - 10uF (0805)
 - 100uF (0805)
- Diodes
 - Red (0603)
 - Green (0603) (2)
- Switches and Buttons

- 01x02 Molex (Case Switch) (3)
- 6mm THM Push Button (2)

4.3.7 Algorithm

```

Initialize:
Configure GPIOs (DATA, CLK, LAT, OE, row select lines (A,B,C,D))
Initialize interface for incoming data
Initialize timer for refresh interrupts
Set PWM resolution (e.g., 8-bit)

Main Loop:
while (true):
    if new frame data available from main PCB:
        copy frame buffer into local memory (double buffer optional)

    for each row_index in 0 .. NUMROWS-1:
        select_row(row_index)
        OE = HIGH

        for pwm_bit = 7 downto 0:    // for 8-bit PWM
            for col_index = 0 .. NUMCOLS-1:
                pixel = frame_buffer[row_index][col_index]
                if pixel.red & (1 << pwm_bit) != 0:
                    DATA_R = HIGH
                else:
                    DATA_R = LOW
                if pixel.green & (1 << pwm_bit) != 0:
                    DATA_G = HIGH
                else:
                    DATA_G = LOW
                if pixel.blue & (1 << pwm_bit) != 0:
                    DATA_B = HIGH
                else:
                    DATA_B = LOW

            pulse(CLK)

        pulse(LAT)
        OE = LOW
        delay(PWMDELAY[pwm_bit])

```

repeat indefinitely

4.3.8 Theory of Operation

The Display subsystem serves as the primary interface between the ESP32-S3-WROOM-1 microcontroller and a HUB75d 32×64 RGB LED matrix, enabling real-time visualization of aviation and weather data. The ESP32 handles pixel generation, frame buffering, and timing control for row and column scanning of the display. Core signals — including LAT (latch), OE (output enable), CLK (clock), and address lines A–E — are driven directly from the ESP32's GPIO pins and level-shifted from 3.3 V logic to the 5 V domain required by the HUB75 interface. The RGB data lines (R1, G1, B1, R2, G2, B2) are similarly translated to ensure proper color channel switching for both the upper and lower halves of the LED matrix.

During operation, the ESP32 continuously cycles through the address lines, updating one row at a time while controlling brightness through pulse-width modulation synchronized with the OE signal. Power is supplied through regulated 5 V and 3.3 V rails, supporting both the LED panel and logic circuitry. Optional pushbuttons provide manual reset and boot functionality, while status LEDs indicate system power and data activity. In system deployment, the display subsystem retrieves aircraft position and weather telemetry from networked sources and ADS-B data, translating this incoming data into text output or images. This modular design allows the subsystem to function independently or as an integrated component of the larger Aviator platform, focusing on robustness, timing precision, and electrical isolation between logic and display domains.

4.3.9 Specifications Measurement

1. Feature a 2048-pixel Dot Display (64 x 32px)
2. The correct screen was ordered, featuring a 32x64px RGB LED matrix
3. Physical dimensions of housing within 5% of 64 x 128 x 256 mm.
4. Measured with ruler and calipers, the actual screen dimensions are 127 x 255 mm.
5. No noticable screen tear (refresh rate $\geq$ 60Hz)
6. In observing the screen during normal operation, the image is clear and constant with no noticable tear. Achieved $\geq$ 60Hz refresh rate

4.3.10 Standards

- **IEC 61010:** Safety for low-voltage electronic equipment; ensures protection against shorts, overcurrent, and handling risks.
- **SPI/I²C:** If MCU/controller communicates with peripheral ICs over standard buses.
- **HUB75:** Standard for 32×64 RGB LED matrices; timing, row multiplexing, and data latching must be followed.
- **JESD51:** Important for high-current LED arrays; ensures heat dissipation and junction temperatures are within safe limits.

4.4 Subsystem 4: Network and Communications

4.4.1 Subsystem Diagrams

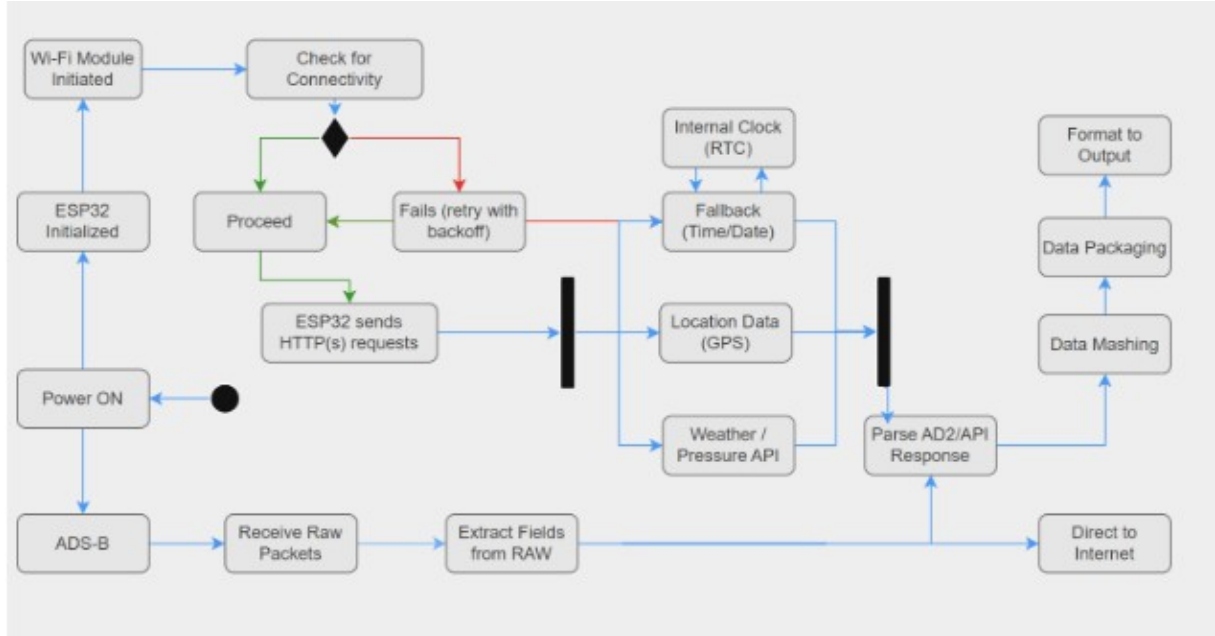


Figure 4.4.1: Network subsystem Block Diagram

4.4.2 Specifications

1. Connection to WPA2 network established in less than 10 s.
2. HTTPS request latency less than 500 ms for payloads up to 2 kB.
3. Outbound HTTP POST/GET success rate at least 95%.
4. Fallback mode automatically activates within 5 s if there is no connectivity.
5. Error handling: up to 3 retries with exponential back-off before fallback.
6. System connectivity uptime at least 95% during a continuous 1-hour runtime.
7. Operates at 1090 MHz Extended Squitter (1090ES) frame format.
8. Reception rate at least 100 packets/s when aircraft are in range.
9. Typical data rate: around 1000 ADS-B frames per second (raw stream).

10. Range: at least 200 nautical miles (line-of-sight).
11. Latency: less than 100 ms for frame decoding.
12. Error rate: less than 1% decode errors with comprehensive error logging.
13. Provides 1Hz fix rate for accurate localization.
14. Supplies location and time data to synchronize ADS-B and API results.

4.4.3 Subsystem Interactions

The communications subsystem acts as the Aviator’s data bridge between external information sources and the internal electronics. It collects ADS-B, API-airport data, and weather data, then forwards validated information to the Processing Subsystem via serial or SPI communication for formatting and display output.

Power is provided by the Power Management Subsystem through a regulated 3.3 V rail, ensuring stable operation of the ESP32 and SDR receiver. In return, the communications unit reports basic voltage or connection status when required.

Externally, this subsystem interacts with aircraft transponders (ADS-B at 1090 MHz), the Open-Meteo API through HTTPS, and other APIs for airport information. Combined, these links guarantee the Aviator always has synchronized flight and environmental data for visualization on the display subsystem.

4.4.4 Core ECE Design Tasks

- **ECE 36200:** Programming the ESP32 for SPI/UART communication and Wi-Fi control, configuring serial buses (UART, SPI), and managing real-time I/O between Wi-Fi, GPS, and ADS-B modules.
- **ECE 43800:** Understanding ADS-B packet decoding and signal validation, including digital filtering, sampling, and signal integrity techniques used during packet extraction.
- **ECE 54700:** Provided with a quantitative understanding of network protocols, error recovery, congestion control, and routing, which directly inform the subsystem’s retry logic, exponential back-off, and HTTP/TCP communication design.
- **ECE 20875:** Implementing data parsing and JSON handling for API responses, plus everything in regard to how to code the process in python.

4.4.5 Schematics

[Type here **DD2+**]

4.4.6 Parts

- **ADS-B Receiver (1090 MHz SDR Dongle):** Captures live aircraft broadcast signals (position, altitude, velocity, ICAO ID) using Automatic Dependent Surveillance–Broadcast (ADS-B). Interfaces with the ESP32 via UART or USB-serial bridge
- **Weather API Module:** Software interface for retrieving real-time weather conditions (temperature, wind, pressure, etc.) through HTTPS requests to the Open-Meteo API.
- **Wi-Fi Network Interface (ESP32 Integrated):** Enables wireless internet connectivity for API calls, data uploads, and synchronization with external services. Supports WPA2 and TCP/IP stack.
- **Data/Signal Interface:** Internal software and pathways responsible for merging ADS-B, GPS, and API data. Manages data transfer to the Processing Subsystem via SPI or serial communication
- **GPS/Geolocation Unit (NEO-6M if used):** Provides precise latitude, longitude, and time data at 1 Hz. Used for correlating aircraft positions and providing fallback localization when ADS-B data are unavailable.

4.4.7 Algorithm

This Subsystem operates through a multi-threaded architecture that concurrently handles ADS-B signal processing and weather data acquisition, ensuring continuous data availability and synchronization between local and online sources.

ADS-B Processing Pipeline

1. **Signal Acquisition:** Continuously monitors the 1090 MHz frequency band for Extended Squitter (DF17) transmissions from aircraft transponders.
2. **Frame Validation:** Each 112-bit frame undergoes length verification and CRC-24 checksum validation to ensure data integrity.
3. **Message Classification:** Extract the Type Code (TC) from bits 32–37 to determine message content:
 - (a) TC 1–4 → Aircraft identification containing 8-character callsign.
 - (b) TC 9–18 → Airborne position with barometric altitude and encoded latitude/longitude.
 - (c) TC 19 → Velocity vectors providing ground speed and track heading.

4. Position Decoding: For location messages, implement the Compact Position Reporting (CPR) algorithm, requiring even/odd frame pairs per aircraft to resolve global coordinates.
5. Database Management: Maintain active flight records indexed by ICAO address, updating position, altitude, and velocity as new frames arrive.

Weather Integration Process

1. Periodic API Calls: Issue HTTPS requests to the Open-Meteo service every 15 minutes for current atmospheric conditions.
2. Multi-Level Data: Retrieve both surface weather and upper-atmosphere conditions at pressure levels (850 hPa – 250 hPa) corresponding to aircraft cruising altitudes.
3. Spatial Correlation: Match weather data to flight positions, providing contextual meteorological information for each tracked aircraft.

Flight Database Correlation (Future Enhancement)

1. Callsign Lookup: Match decoded aircraft callsigns with commercial flight-tracking APIs such as FlightAware or OpenSky Network.
2. Route Information: Correlate ICAO addresses with scheduled flight data to determine origin–destination pairs, delays, and gate assignments.
3. Enhanced Display: Transform raw aircraft data (position, altitude) into meaningful flight information (route, delay status, aircraft type).

Data Fusion and Export

1. Distance Calculation: Apply the formulas required to compute aircraft proximity to the receiver’s GPS location.
2. Error Handling: Implement comprehensive logging and retry mechanisms for malformed frames, failed API calls, and decoding errors.
3. Interface Abstraction: Export processed data through high-level APIs from functions that fetch flight-weather info that encapsulate protocol complexity from downstream subsystems.

Real-Time Operation: Maintaining a continuous operation via non-blocking frame processing, periodic weather updates, automatic database pruning of stale aircraft records, and dual-mode functionality. This will ensure reliable data flow even during bad network or RF interference.

4.4.8 Theory of Operation

Upon initialization, the ESP32 microcontroller establishes WiFi connectivity and begins continuous monitoring of the 1090 MHz ADS-B frequency band. The ADS-B receiver captures 112-bit Extended Squitter (DF17) frames broadcast by aircraft transponders, containing encoded ICAO identifiers, position data, altitude, and velocity information. Each frame undergoes CRC validation and bit-level decoding, extracting Type Codes for identification (TC 1-4), airborne position (TC 9-18), and velocity (TC 19). Position data uses Compact Position Reporting (CPR) algorithms, requiring even/odd frame pairs to decode latitude/longitude coordinates.

Simultaneously, the system issues periodic HTTPS requests to the Open-Meteo weather API, retrieving both surface conditions and upper-atmosphere data at pressure levels (850hPa to 250hPa) corresponding to aircraft cruising altitudes. Weather data is correlated with flight positions to provide contextual information. Additionally, information from airport information is looked for to match with ADS-B information about planes caught by the antenna, meaning one can match data and determine if flights are delayed, origin-destination, etc.

The ESP32 processes all incoming data streams in real-time, maintaining an active flight database indexed by ICAO address. Distance calculations use formulas to determine aircraft proximity to the receiver. Comprehensive error handling includes frame validation, API timeout recovery, and statistical error tracking to ensure robust operation.

4.4.9 Specifications Measurement

[DD3+ Every specification here should match the specification above.]

1. [Copy specification here.]
[Explain the specification here. Add photos if necessary.]

4.4.10 Standards

- **NMEA 0183:** Used by GPS receivers to format location/time data, ensuring compatibility with processing algorithms.
- **REST/HTTPS:** Secure, standardized communication with weather servers and external data sources.
- **IEEE 802.11 (Wi-Fi) / IEEE 802.3 (Ethernet):** Provides reliable networking for internet-based data exchange (when we add website interference).
- **TCP/IP, Checksum validation:** Ensures robustness in communication so that even non-technical users get accurate, reliable results.



Figure 4.4.2: [Schematic Name]

5 PCB Design

5.1 PCB Schematics

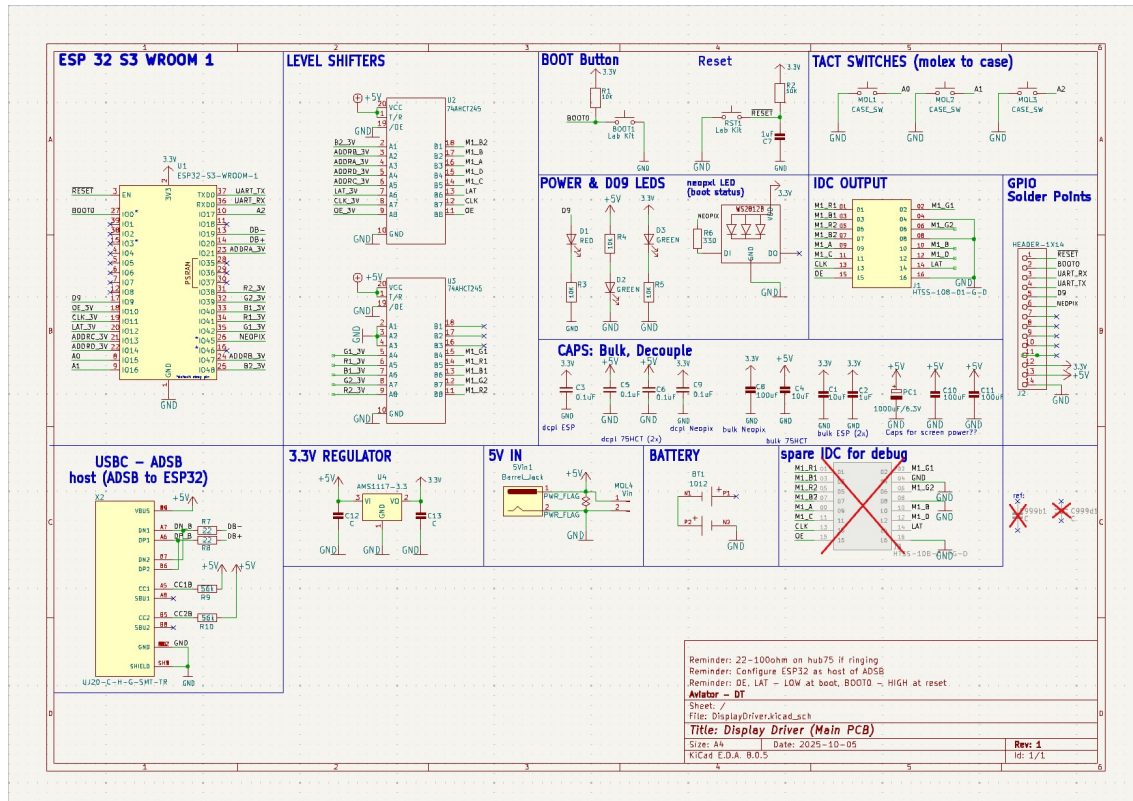


Figure 5.1.1: PCB Schematic

5.2 PCB Layout

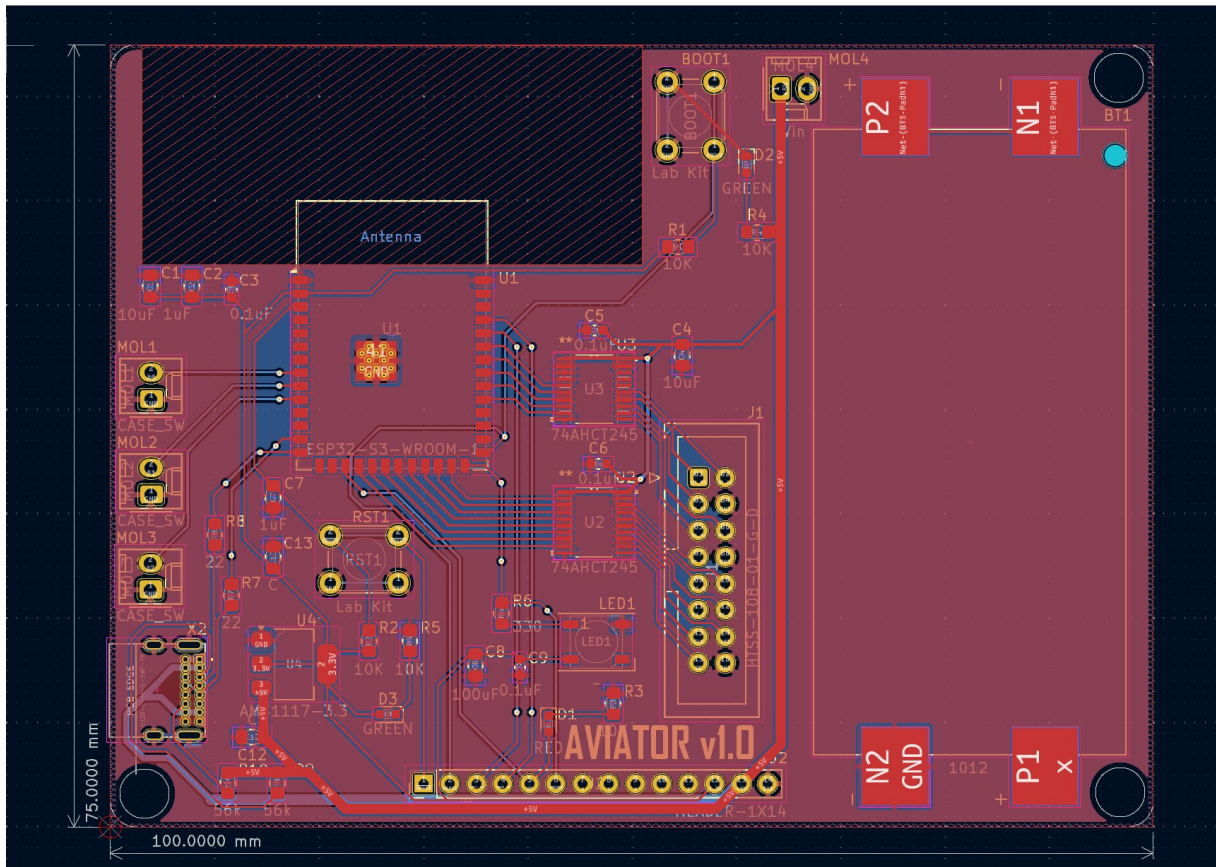


Figure 5.2.1: PCB Layout

6 Final Status of Requirements

1. Requirement 1: The device must display accurate information.
Met: all information displayed is timestamped and discarded once stale. The device is paired with a local ADS-B receiver to ensure coverage of local flight since open flight APIs lack providers in most areas, ensuring overhead flights are accurately captured on-screen.
2. Requirement 2: The display must not interfere with user's well-being by, for example, displaying at excessive luminosity, updating rapidly in a distracting manner, or being excessively bulky.
Met: luminosity is configurable, animations are limited to scrolling overflowed text, and the device is contained within a 10x5x3 cubic inch volume.
3. Requirement 3: The device must not infringe on any person's reasonable expectation of privacy.
Partially met: the device's location could be determined by anyone with access to the API by plotting the epicenter of the newly appearing flights, and arguably tracking flights violates the privacy of flyers, but overall the device leaks very little personally identifiable information.
4. Requirement 4: The device must be language-agnostic wherever possible.
Not met: the interface is in English due to the difficulty of displaying discernible language-agnostic icons on a screen with 32x64 resolution. The documentation is also English.
5. Requirement 5: The device must be responsive and intuitive.
Met: the user interface is limited to swapping in between different display modes. All other configuration is automatic. The device responds in a fraction of a second to user input.
6. Requirement 6: The device must have robust error handling and recovery.
Met: the device is capable of autonomously recovering from any software error.
7. Requirement 7: The physical device should be easily replicated with widely available parts.
Met: by design, every component of the device is affordably sourced, with no custom machining needed beyond the PCB. A hobbyist or electrical engineering student should be able to recreate the device in a few hours.
8. Requirement 8: The code for the device must be open-source and well-documented.
Partially met: the code and PCB designs are published with open-source licenses. However, the documentation is unaligned with good practice and lacking in several areas.

9. Requirement 9: The device should be as durable and environmentally friendly as possible so as not to contribute to e-waste.
Partially met: the device is designed to last a minimum of four years of continuous operation, but the components are not sourced sustainably, and construction is not eco-friendly.
10. Requirement 10: The device must not contribute to noise or visual pollution of any space.
Met: the device emits no sound and displays no visually disrupting imagery.
11. Requirement 11: The device must be energy-efficient.
Partially met: the display consumes approximately 1.3 watts during normal operation, 16% of the maximum possible power consumption of the RGB LED display. While not low-power, the device therefore does use what power is available conservatively.
12. Requirement 12: The device must minimize construction and recurring costs.
Met: the device costs approximately \$110 USD to construct, including the ADS-B SDR. By using only free APIs, the device incurs no recurring costs apart from energy use.
13. Requirement 13: The device must not infringe on right to repair.
Met: all information needed to construct, maintain, and repair the device is freely available.
14. Requirement 14: The device must mount and dismount without damage to vertical surfaces.
Met: through magnets embedded in the casing, the device can attach to magnetic surfaces without adhesive. The back of the device is also conducive to non-damaging adhesive strips.

7 Team Structure

7.1 Team Member 1



David Thoe

Major: Electrical Engineering

Contact: dtthoe@purdue.edu

Team Role: Team Leader

Bio: In charge of graphics drivers and text display, I will be focused on the final presentation of the information as well as subsystem integration. I will also be paying special attention to the housing of the prototype, aided by CAD and 3D printing, ensuring the product appears polished when complete. At Purdue, I concentrate in Wireless and Optical engineering and participate in the club Autonomous Motorsports Purdue, where we place our fully autonomous go-kart in competition with other universities. In my free time, I work on hobby electrical projects usually related to my computer or decoration.

7.2 Team Member 2



Joshua Kim

Major: Electrical Engineering

Contact: kim3503@purdue.edu

Team Role: Communication Lead

Bio: In my role as Communication Lead, I am responsible for ensuring that communication on our team, with GTA, and the Professor is clear and consistent. I manage communications on updates, facilitate collaboration on progress reporting, and keep everyone engaged

and aligned within the subsystem groups to further our design as a cohesive project. My background in Electrical Engineering enables me to contribute directly to the technical aspects of the project and also help translate technical details into clear explanations for our team and other stakeholders. Balancing the communication and development of the subsystem, I can keep both the engineering work and project coordination rolling smoothly.

7.3 Team Member 3



Zeke Ulrich

Major: Computer Engineering

Contact: pulrich@purdue.edu

Team Role: Treasurer

Bio: Zeke is the processing and PCB design specialist. He's responsible for designing the PCB schematics and integrating the disparate subsystems on the processor. At Purdue, Zeke belongs to the Marine Corp Officer Candidate Program, Eta Kappa Nu, Tau Beta Pi, and Purdue's Effective Altruism community. Outside Purdue, he is president of the nonprofit DuelGood and works for the government in cybersecurity. In the future he hopes to study international relations as a Truman scholar, start a family, and volunteer as a firefighter. He enjoys athletics and spending time with his friends.

7.4 Team Member 4



Juan Vargas

Major: Electrical Engineering

Contact: varga105@purdue.edu

Team Role: Facilitator

Bio: As the Facilitator, my primary role is to keep our team organized, collaborative, and on track throughout the project. I focus on making sure discussions are productive, tasks are clearly divided, and deadlines are met without overwhelming any single member. By bridging technical conversations and helping resolve roadblocks quickly, I ensure that progress stays steady and balanced across subsystems. Alongside this coordination, I contribute to the technical development by assisting with software and system integration specially in the network/communication part, ensuring our device works as intended while maintaining the retro, aviation-inspired feel that defines Aviator.

8 Bibliography

[Here are some examples. IEEE format can be found on [Purdue OWL](#).]

References

- [1] “Data Platform - Open Power System data,” Apr. 15, 2020. https://data.open-power-system-data.org/household_data/
- [2] Author, ”Title,” *Journal*, volume, number, page range, month year, DOI.
- [3] Author. ”Page.”Website. URL(accessed month day,year)

9 Appendices

[This section is mainly designed for code. You can directly generate a somewhat decent display of your code file or psuedo code by using the template provided below. You can have as many appendix as you want. In the document, you can refer to the code posted here instead of pasting the whole code in the body.]